Introduction to CSS

CSS Resets

Every browser ships with a "user-agent stylesheet," i.e. built-in default styles that give headings, paragraphs, lists, and form controls their initial look. These defaults differ slightly between browsers (and sometimes between versions). Historically, the differences were much, much worse.

So, a **CSS reset** is a small stylesheet you include first to standardize those defaults so that your own styles start from a predictable baseline. Here's a simple example:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

Everything is a box!

You can see for yourself by adding `* { outline: auto }` to the CSS of almost any webpage.

Or, if you're feeling brave, run this Javascript snippet in your browser's console (you normally shouldn't copy/paste code from the internet into your browser's console, but... you can trust me on this one):

```
(()=>{const e=(()=>{let e=0;const t=(n,i)=>{e=Math.max(e,i);for(let e of n.children
```

The box model isn't unique to CSS. Most visual software treats things as rectangles for layout and collision. A fighting game decides if a punch lands by checking overlapping hitboxes; a mobile app decides if a tap counts by checking a rectangular touch frame. Even vector graphics and PDFs place objects inside bounding rectangles.

All boxes have these CSS properties

- `width` and `height`
- `margin`
- `padding`
- `border`

- `border-radius`
- `background`
- `box-shadow`

You can also set `max-width` and `max-height` to limit the size of a box.

A small nuance: inline elements technically can have these properties, but top/bottom margins often have no effect, and width/height aren't honored unless you change their display type.

There are various units you can use

- Pixels: `16px`
- Percentages: `50%`
- Viewport units: `100vw` or `100vh`
- Em and rem: `1em` or `1rem`

The value of `auto` can be used with `margin` to center a box horizontally. Like `margin: 0 auto;`. But your box will need a defined width for that to work. Like this:

```
.centered {
  width: 200px;
  margin: 0 auto;
}
```

We'll learn other centering techniques later, but the ol' `margin: 0 auto;` is simple and sufficient for now.

Shorthand syntax

- Defining all four sides: `margin: 10px;`
- Defining top/bottom and left/right: `padding: 10px 20px;`
- Defining top, left/right, and then bottom: `margin: 0 auto 20px;`
- Defining each side individually: `padding: 10px 20px 30px 40px;`

Shorthand syntax for borders

- Defining all four sides: `border: 1px solid #000;`
- Defining each side individually: `border-top: 1px solid #000;`, `border-right: 1px solid #000;`, etc.

You are also define the width of each side individually using `border-width`, if you also have `border-style` and `border-color` set.

```
.weird-border {
  border-style: solid;
  border-color: #000;
  border-width: 4px 8px 16px 32px;
}
```

Styling

Almost all styling comes down to those same few box properties

- `width` and `height`
- `margin`
- `padding`
- `border`
- `border-radius`
- `background-color`
- `box-shadow`

Plus typography, which we will start getting into next class... But for now:

- `font-family`
- `font-size`
- `text-transform`
- `text-align`
- `color`

Pseudo-classes

There are special selectors that are used to capture special states of an element.

- `:link` when a link has not been visited.
- `:visited` when a link has been visited.
- `:hover` when you mouse over an element, commonly used for buttons and links, but could be used for any element (table row is another common one).
- `:focus` when an element is selected, commonly used for form inputs.

There are also pseudo-classes for form inputs:

- `:checked` when a checkbox or radio button is checked.

- `:valid` when a form input is valid.
- `:invalid` when a form input is invalid.

And pseudo-classes for groups of elements:

- `:first-child` when an element is the first child of its parent.
- `:last-child` when an element is the last child of its parent.
- `:nth-child()` when an element is the nth child of its parent.

`:nth-child()` can take a number, like `:nth-child(3)`, or a formula, like `:nth-child(2n)` or `:nth-child(3n+1)`. Most commonly, it is used to stripe tables with `:nth-child(odd)` and `:nth-child(even)`.

Positioning Properties

- `position: static;` (default)
- `position: relative;`
- `position: absolute;`
- `position: fixed;`
- `position: sticky;`

`sticky` is like a mix between `relative` and `fixed`. The element is treated as `relative` until it crosses a specified threshold, then it becomes `fixed`. Try it out!

A new way to layout elements

Flexbox is a layout model that allows you to design complex layouts with a single container element and its children. It's a powerful tool that makes it easier to create responsive designs and align elements in a predictable way.

Basic usage

To use Flexbox, you need to define a `display: flex;` property on the parent container. This will turn the container into a flex container and allow you to use flex properties to control the layout of its children.

```
<!-- HTML -->
<div class="container">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <div class="item">Item 3</div>
</div>
```

```css
/* CSS */
.container {
  display: flex;
}
```

Flex properties for the container

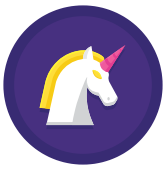
There are several properties you can use to control the layout of the flex container:

- `flex-direction`: defines the direction of the main axis
- `justify-content`: aligns items along the main axis
- `align-items`: aligns items along the cross axis
- `flex-wrap`: controls how items wrap
- `align-content`: aligns items when there is extra space in the cross axis
- `gap`: defines the space between items

Here's a demo showing a typical flexbox use case of a navigation bar:

```html
<!-- HTML -->
<nav class="navbar">
  <svg>...</svg>
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Services</a>
  <a href="#">Contact</a>
</nav>
```

```css
/* CSS */
.navbar {
  display: flex;
  justify-content: flex-start;
  align-items: center;
  gap: 0.25rem;
}
```



Home About Services Contact

Flex properties for the children

There are also properties you can use to control the layout of the flex items:

- `flex-grow`: how much an item will grow relative to the other items
- `flex-shrink`: how much an item will shrink relative to the other items
- `flex-basis`: the initial size of an item
- `order`: the order of the item in the flex container
- `align-self`: allows an item to override the `align-items` value for its container

Here's an example using the `order` property to move the second item to the beginning:

```
<!-- HTML -->
<div class="container">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <div class="item">Item 3</div>
</div>

/* CSS */
.container {
  display: flex;
}

.item:nth-of-type(2) {
  order: -1;
}
```

Item 2 Item 1 Item 3

Here's an example using the `flex-grow` property to make the first item grow to fill the available space:

```
.item:first-of-type {
  flex-grow: 1;
}
```

Item 1 Item 2 Item 3

As a shorthand, you can use the `flex` property to set the `flex-grow`, `flex-shrink`, and `flex-basis` values at once. Some examples:

- `flex: 1;` will make an item grow to fill the available space.

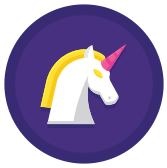
- `flex: 0 1 200px;` will make an item not grow, shrink to fit, and have an initial size of 200px.

```
.item {
  flex: 1;
}
```

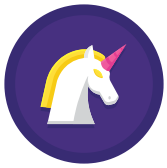
Item 1 Item 2 Item 3

A few different nav examples

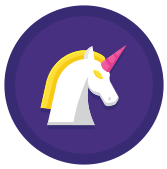
Here are a few different examples of navigation bars using flexbox. Use your browser's dev tools to inspect the elements and see how they work.



Home About Services Contact



Home About Services Contact



Home About Services Contact

Media Queries

Media queries are a way to apply CSS rules based on the characteristics of the device the site is being displayed on. For example, you can use media queries to apply different styles for small screens, large screens, or even for printing.

The most common media queries are `min-width` and `max-width`. You can also use `orientation` to apply styles based on the orientation of the device (portrait or landscape).

```
@media (max-width: 600px) {
  /* styles here */
```

```
}

@media (orientation: landscape) {
  /* styles here */
}
```

For accessibility, you can also use `prefers-color-scheme` to apply a light or dark theme based on the user's preferences, and `prefers-reduced-motion` to reduce animations for users who prefer less motion.

```
@media (prefers-color-scheme: dark) {
  /* styles for dark mode */
}

@media (prefers-reduced-motion: reduce) {
  /* styles for reduced motion, e.g. turning off animations */
}
```

You can combine media queries to create more complex rules. For example, you can apply styles for screens under 600px wide in landscape orientation:

```
@media (max-width: 600px) and (orientation: landscape) {
  /* styles here */
}
```

Psuedo-elements

Like pseudo-classes, pseudo-elements are used to style specific parts of an element. They are denoted by a double colon (`::`) instead of a single colon (`:`).

Some common pseudo-elements include:

- `::before` - inserts content before the element
- `::after` - inserts content after the element
- `::first-line` - styles the first line of text
- `::first-letter` - styles the first letter of text

And for form inputs:

- `::placeholder` - styles the placeholder text in an input field

Here's an example of using `::first-letter` to create a drop cap effect:

```
.drop-cap::first-letter {
  font-size: 2.75em;
  font-family: "Times New Roman", serif;
  float: left;
  line-height: 0.8;
  margin-right: 0.1em;
  color: #113f67;
}
```

This paragraph demonstrates the drop cap effect using the ::first-letter pseudo-element. Notice how the first letter is larger and uses a fancy font. Blah blah blah some more text here.

And here's an example of styling a placeholder:

```
input::placeholder {
  color: blue;
  font-style: italic;
  font-size: 0.9em;
}
```

Enter your name...

Nesting

Nesting is a modern feature that allows you to write easier to read, more modular, and more maintainable CSS. As you are not constantly repeating selectors, the file size can also be reduced.

For example, you can style both a `<ul>` and its `<li>` children like this:

```
ul {
  list-style-type: none;

  li {
    padding: 10px;
  }
}
```

Be careful not to overuse it, as it can lead to overly specific selectors and make your CSS harder to override.

Variables

Variables are a way to store and reuse values in your CSS. They can be used to store colors, font sizes, margins, and more. This can make your CSS more maintainable and easier to update.

To define a variable, use the `--` prefix:

```
:root {
  --primary-color: #007bff;
}

.button {
  background-color: var(--primary-color);
}
```

The `:root` selector is used to define global variables that can be accessed anywhere in your CSS. You can also define variables within a specific selector:

```
.button {
  --primary-color: #007bff;

  background-color: var(--primary-color);
}
```

Functions

CSS functions allow you to perform calculations, manipulate colors, and more. Some common functions include `calc()`, `rgb()`, `rgba()`, `hsl()`, `hsla()`, and `var()`.

For example, you can use the `calc()` function to perform calculations:

```
.container {
  width: calc(100% - 20px);
}
```

Or use the `rgba()` function to specify a color with an alpha channel:

```
.button {
  background-color: rgba(0, 123, 255, 0.5);
}
```

Filters

CSS filters allow you to apply visual effects to elements, such as blurring, color shifting, and more. Some common filters include `blur()`, `brightness()`, `contrast()`, `grayscale()`, `invert()`, `opacity()`, `saturate()`, and `sepia()`.

For example, you can use the `blur()` filter to create a blurred effect:

```
.image {
  filter: blur(5px);
}
```

Or use the `grayscale()` filter to create a grayscale effect:

```
.image {
  filter: grayscale(100%);
}
```

Here's an image with both of those applied:

Transformations

Transformations allow you to modify the appearance of an element. Some common transformations include `rotate()`, `scale()`, `skew()`, and `translate()`.

```
.box {
  transform: rotate(-15deg) skew(10deg);
}
```

Less common transformations include `rotateX()`, `rotateY()`, and `rotateZ()` for rotating around specific axes; `matrix()` and `matrix3d()` for more complex transformations; and `perspective()` for creating a 3D perspective effect.

Transitions

Transitions allow you to animate changes in CSS properties. You can specify the duration, timing function, delay, and more. Some common properties to transition include `color`, `background-color`, `opacity`, `transform`, and more.

The rule of thumb is that the browser can animate properties that are numbers, colors, or lengths (basically anything that can be interpolated). If you try to animate a property that can't be interpolated, the browser will not animate it.

```
.button {
  transition: background-color 0.3s ease;
}

.button:hover {
  background-color: #0056b3;
}
```

Hover over me

Notice that the `transition` property is applied to the base state, and the change in the hover state triggers the transition. This creates a smooth animation effect when the background color changes.

You can also use the `transition` property to animate multiple properties at once. Just separate them with a comma like `transition: background-color 0.3s ease, color 0.3s ease;`

Animations

Animations allow you to create more complex and dynamic effects in CSS. You can define keyframes to specify the animation's behavior, such as the starting and ending states, and the timing of the animation.

Here's an example of a simple animation that moves an element from left to right:

```
@keyframes slide {
  0% {
    transform: translateX(0);
  }

  100% {
    transform: translateX(100px);
  }
}

.box {
  animation: slide 2s infinite;
}
```

The `@keyframes` rule defines the animation behavior, and the `animation` property applies the animation to that specific element.

For readability, you can also write `from` and `to` instead of `0%` and `100%`:

```
@keyframes slide {
  from { transform: translateX(0); }
```

```
  to   { transform: translateX(100px); }
}
```

Animations can be used to create a wide variety of effects, from simple transitions to complex interactive experiences. They can be combined with other CSS properties like transforms, transitions, and filters to create visually stunning effects.

Transitions vs Animations

Transitions and animations are both used to create motion effects in CSS, but they have different use cases:

Transitions are best for simple effects like hover states or smooth property changes. They require a user interaction to trigger the animation (like `:hover`). They are easier to set up and require less code.

Animations are better for complex effects that involve multiple keyframes and more advanced timing functions. They offer more control and flexibility but require more code to set up.

Resources

- MDN Web Docs: Basic Concepts of Flexbox
- CSS-Tricks: A Complete Guide to Flexbox
- Flexbox Froggy: A game to learn Flexbox